

Fundamentals of Deep Learning and Programming

Lecture 11: Diffusion Models

Fall Semester 2025

Adjunct Professor: Donghoon Kang

Term Project

Due date: (12.12, Friday)

Should **submit** an **5 ~ 6 page report** as a term-project.

Extention to the term project
submission deadline is permitted
until 11 PM on 12.21(Sunday)

December						
Su	Mo	Tu	We	Th	Fr	Sa
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

UST Course Fall Semester 2025

Fundamentals of Deep Learning and Programming

1. **Week 1 (9.05):** Course overview [pdf](#) and introduction to Python programming [pdf](#)
2. **Week 2 (9.12):** Useful Python Libraries [pdf](#) and code [zip](#)
3. **Week 3 (9.19):** Neural Network Basics (1) [pdf](#) and code [zip](#)
4. **Week 4 (9.26):** Neural Network Basics (2) [pdf](#) and code [zip](#)
5. **Week 5 (10.10):** Convolutional Neural Network (CNN) (1) [pdf](#) and code [zip](#)
6. **Week 6 (10.24):** Midterm exam [pdf](#) and the solution [pdf](#)
7. **Week 7 (10.31):** Convolutional Neural Network (CNN) (2) and Introduction to Natural Language Processing (NLP) (o) [pdf](#)
8. **Week 8 (11.14):** Introduction to Natural Language Processing (NLP)(1) [pdf](#) and code [zip](#)
9. **Week 9 (11.21):** Introduction to Natural Language Processing (NLP)(2) & Attention Mechanism [pdf](#) and code [zip](#)
10. **Week 10 (11.28):** Transformer (1) [pdf](#) and code [zip](#)
11. **Week 11 (12.05):** Transformer (2) [pdf](#)

Two papers for final exam preparation: "Attention is all you need" (NeurIPS 2017) [pdf](#) and "An image is worth 16x16 words" (ICLR 2021) [pdf](#)

Term Project Reports

- Mr. Suleman Khan [pdf](#) ("EMGBench: Benchmarking Out-of-Distribution Generalization and Adaptation for Electromyography", NeurIPS 2024) [pdf](#)
- Mr. JeongMin Heo [pdf](#) ("NightReID: A Large-Scale Nighttime Person Re-Identification Benchmark", AAAI 2025) [pdf](#)

English Name

Course Overview

Lecture Time

Fridays, 2 PM – 5 PM

Classroom

Room: L3 301, 302 (3334A, B)

Evaluation Criteria

Attendance (10%)

Programming Assignments (5%)

Term-project (15%)

Mid-term (30%), Final (40%)

Final Exam

Week 16 (12.19, Friday)

Final exam covers **all the material (=everything)** learned in class, including the midterm exam.

Meaning of the Expectation $\mathbb{E}_{x \sim p_{\text{data}}}[\cdot]$

The notation

$$\mathbb{E}_{x \sim p_{\text{data}}}[\cdot]$$

means an **expectation** (average) over data points x that are drawn from the true data distribution $p_{\text{data}}(x)$.

Data Distribution $p_{\text{data}}(x)$

- $p_{\text{data}}(x)$ denotes the (unknown) probability distribution of real data.
- When we write $x \sim p_{\text{data}}$, it means that x is a random variable sampled from this true data distribution.

Expectation as an Integral or Average

Formally, the expectation of a function $f(x)$ under the distribution $p_{\text{data}}(x)$ is defined as:

$$\mathbb{E}_{x \sim p_{\text{data}}}[f(x)] = \int f(x) p_{\text{data}}(x) dx.$$

In practice, we do not know the exact form of $p_{\text{data}}(x)$. Instead, we approximate the expectation by an average over the training dataset $\{x^{(1)}, \dots, x^{(N)}\}$:

$$\mathbb{E}_{x \sim p_{\text{data}}}[f(x)] \approx \frac{1}{N} \sum_{i=1}^N f(x^{(i)}).$$

Kullback–Leibler (KL) divergence is a mathematical tool used to measure how different two probability distributions are.

KL divergence tells you how much one probability distribution disagrees with another.

What is a Probability Distribution?

A probability distribution describes how likely different outcomes are. Examples:

- A fair coin: heads = 50%, tails = 50%.
- A biased coin: heads = 90%, tails = 10%.

If two distributions are exactly the same, there is no disagreement. If they are very different, the disagreement is large.

Suppose:

- P is the *true* distribution.
- Q is the *approximate* distribution.

KL divergence measures how much extra “surprise” you experience when you assume Q is true but the world actually follows P .

KL divergence is small when Q is a good approximation of P .

KL divergence is large when Q is a poor approximation of P .

Imagine you go to a vending machine that *usually* gives:

$$P = \{\text{chips: 70\%, cookies: 30\%}\}.$$

But your friend tells you:

$$Q = \{\text{chips: 20\%, cookies: 80\%}\}.$$

If you trust your friend’s distribution Q , you will expect cookies most of the time. But in reality, the machine gives chips most of the time.

This mismatch in expectation is exactly what KL divergence measures.

The KL divergence from distribution P to Q is:

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}.$$

- $P(x)$: the true probability of outcome x .
- $Q(x)$: the approximate probability.
- $\log \frac{P(x)}{Q(x)}$: the amount of “extra surprise” when using Q instead of P .

Key Intuition

The KL divergence from distribution P to Q is:

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}.$$

- If $P(x) = Q(x)$ for all x , then:

$$D_{\text{KL}}(P \parallel Q) = 0.$$

- KL divergence is always non-negative:

$$D_{\text{KL}}(P \parallel Q) \geq 0.$$

- KL divergence is *not symmetric*:

$$D_{\text{KL}}(P \parallel Q) \neq D_{\text{KL}}(Q \parallel P).$$

It matters which one you call the “true” distribution.

Non-Negativity

KL divergence can be interpreted as the “extra cost” or “extra surprise” that occurs when we use Q to encode or model data that actually follow P . Extra cost cannot be negative, which matches the mathematical result

$$D_{\text{KL}}(P \parallel Q) \geq 0.$$

Asymmetry

The direction matters. The quantity

$$D_{\text{KL}}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)} = \mathbb{E}_{x \sim P} \left[\log \frac{P(x)}{Q(x)} \right].$$

$$D_{\text{KL}}(P \parallel Q)$$

answers the question:

How bad is it to use Q when the true distribution is P ?

By contrast,

$$D_{\text{KL}}(Q \parallel P)$$

answers:

How bad is it to use P when the true distribution is Q ?

Since these questions are different, the numerical answers are also different in general, which explains the asymmetry.

GAN (Generative Adversarial Network)

A Generative Adversarial Network (GAN) is a deep learning framework used to generate new data such as images, audio, or text. GANs are known for producing highly realistic synthetic images and are widely used in modern generative AI.

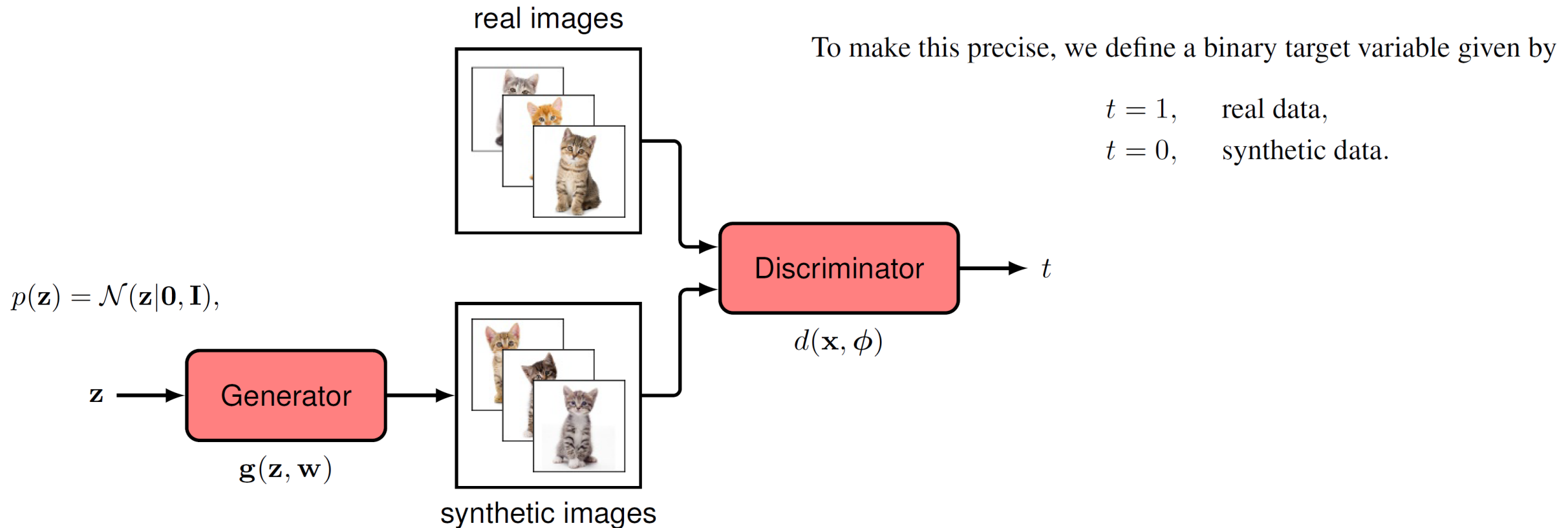
A GAN consists of two neural networks that compete with each other:

Generator $g(z, \mathbf{w})$ and Discriminator $d(x, \phi)$.

Here:

- z is a point in the **latent space** $\mathcal{Z}$,
- x is a point in the **data space** $\mathcal{X}$,
- $\mathbf{w}$ denotes the parameters of the Generator,
- ϕ denotes the parameters of the Discriminator.

GAN (Generative Adversarial Network)



Schematic illustration of a GAN in which a discriminator neural network $d(\mathbf{x}, \phi)$ is trained to distinguish between real samples from the training set, in this case images of kittens, and synthetic samples produced by the generator network $g(\mathbf{z}, \mathbf{w})$. The generator aims to maximize the error of the discriminator network by producing realistic images, whereas the discriminator network tries to minimize the same error by becoming better at distinguishing real from synthetic examples.

The Generator : Creating Fake Data

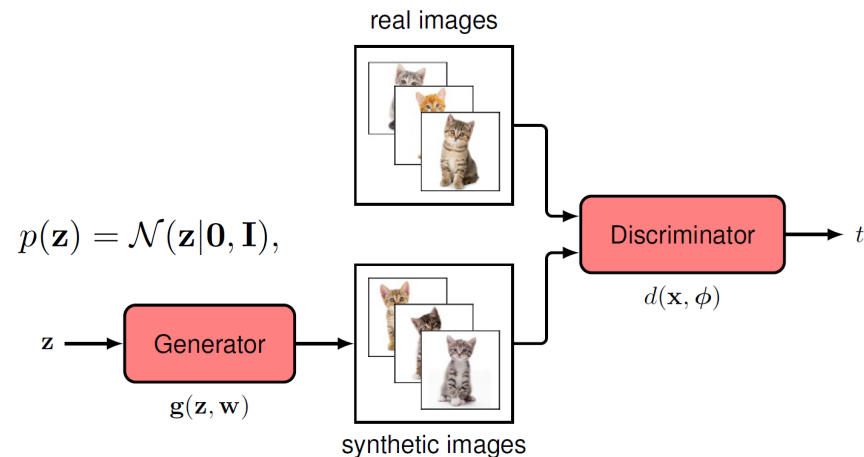
The Generator maps latent vectors $z \in \mathcal{Z}$ to the data space $\mathcal{X}$. Formally, the Generator is a function

$$g : \mathcal{Z} \times \mathbb{R}^{|\mathbf{w}|} \rightarrow \mathcal{X},$$

and given a random noise vector z and parameters $\mathbf{w}$, it produces a synthetic (fake) sample:

$$\hat{x} = g(z, \mathbf{w}).$$

The goal of the Generator is to create data in the data space $\mathcal{X}$ that looks as real as possible, so that the Discriminator cannot distinguish $\hat{x}$ from a true data point x .



To make this precise, we define a binary target variable given by

$$\begin{aligned} t = 1, & \quad \text{real data,} \\ t = 0, & \quad \text{synthetic data.} \end{aligned}$$

The Discriminator takes a data-space input $x \in \mathcal{X}$ (either real or generated) and outputs a scalar between 0 and 1, representing how likely it believes the input is real.

We can write the Discriminator as a function

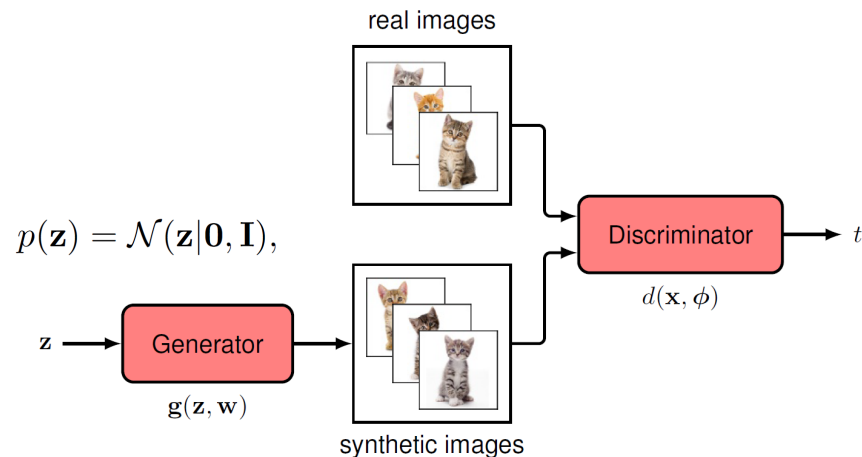
$$d : \mathcal{X} \times \mathbb{R}^{|\phi|} \rightarrow [0, 1],$$

so that, for parameters ϕ ,

$$d(x, \phi) \approx 1 \quad \text{if } x \text{ is real,} \quad d(x, \phi) \approx 0 \quad \text{if } x \text{ is fake.}$$

In particular, for a generated sample $\hat{x} = g(z, \mathbf{w})$, the Discriminator evaluates

$$d(g(z, \mathbf{w}), \phi).$$



To make this precise, we define a binary target variable given by

$$\begin{aligned} t &= 1, & \text{real data,} \\ t &= 0, & \text{synthetic data.} \end{aligned}$$

Adversarial Training Process

GAN training is a minimax game where:

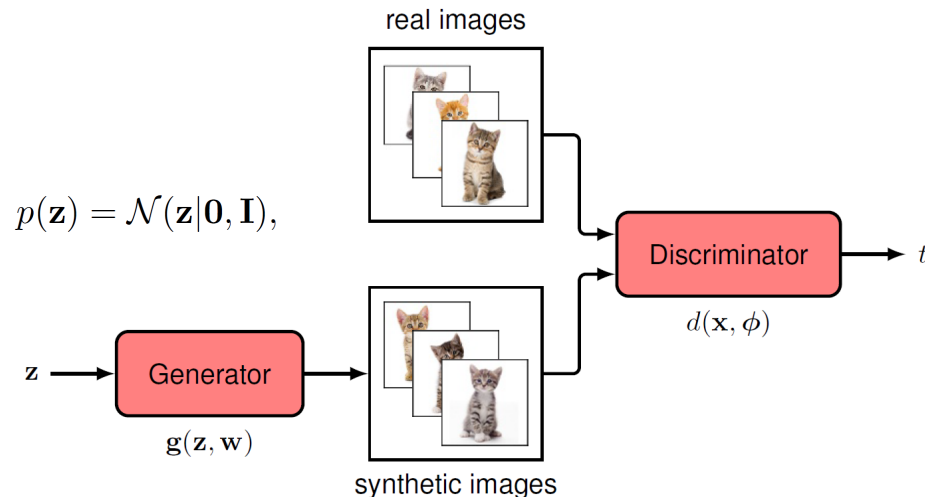
- the Generator (parameters $\mathbf{w}$) tries to fool the Discriminator,
- the Discriminator (parameters ϕ) tries to correctly identify fake samples.

An intuitive analogy:

The Generator $g(z, \mathbf{w})$ is a forger trying to create realistic fake images in the data space $\mathcal{X}$.

The Discriminator $d(x, \phi)$ is a detective trying to detect forgeries.

They both improve through competition.



To make this precise, we define a binary target variable given by

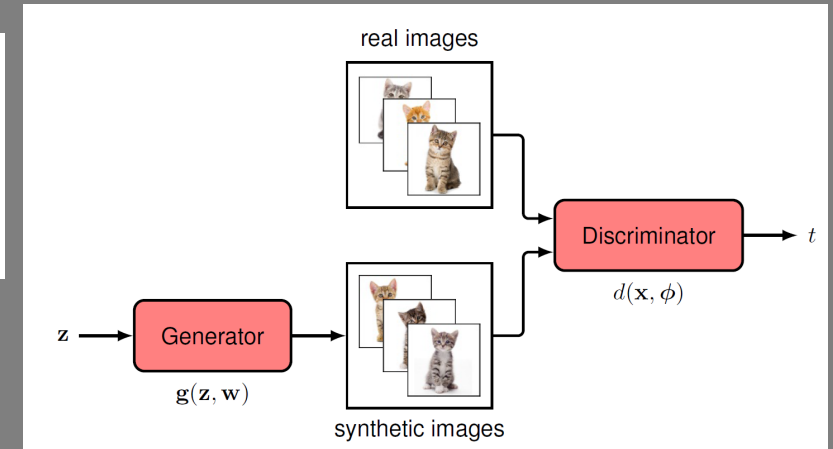
$$\begin{aligned} t &= 1, & \text{real data,} \\ t &= 0, & \text{synthetic data.} \end{aligned}$$

Adversarial Training Process

We denote:

$$g(z, \mathbf{w}) : \mathcal{Z} \rightarrow \mathcal{X}, \quad d(x, \phi) : \mathcal{X} \rightarrow (0, 1).$$

Here, $d(x, \phi)$ outputs the predicted probability that x is real.



Discriminator Loss (Binary Cross-Entropy Form)

The Discriminator is trained as a binary classifier:

- Real data $x \sim p_{\text{data}}$ has label $y = 1$.
- Generated data $\hat{x} = g(z, \mathbf{w})$ has label $y = 0$.

Thus, its loss is the **negative log-likelihood**:

$$\mathcal{L}_D(\phi, \mathbf{w}) = -\mathbb{E}_{x \sim p_{\text{data}}} [\log d(x, \phi)] - \mathbb{E}_{z \sim p(z)} [\log(1 - d(g(z, \mathbf{w}), \phi))].$$

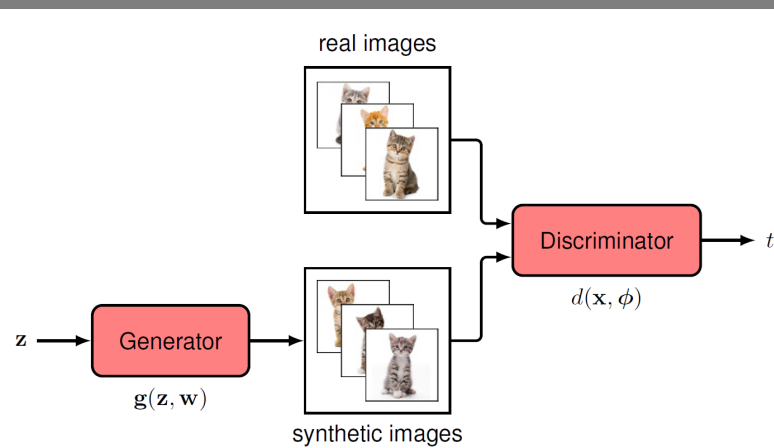
Discriminator Loss (Binary Cross-Entropy Form)

its loss is the **negative log-likelihood**:

$$\mathcal{L}_D(\phi, \mathbf{w}) = -\mathbb{E}_{x \sim p_{\text{data}}} [\log d(x, \phi)] - \mathbb{E}_{z \sim p(z)} [\log(1 - d(g(z, \mathbf{w}), \phi))].$$

- The first term penalizes the Discriminator if it assigns low probability to real samples.
- The second term penalizes the Discriminator if it mistakenly assigns high probability to fake samples.

The Discriminator tries to *minimize* this loss.



Generative Adversarial Networks (GANs) are trained using the Discriminator loss written in negative log-likelihood (binary cross-entropy) form:

$$\mathcal{L}_D(\phi, \mathbf{w}) = -\mathbb{E}_{x \sim p_{\text{data}}} [\log d(x, \phi)] - \mathbb{E}_{z \sim p(z)} [\log(1 - d(g(z, \mathbf{w}), \phi))].$$

The Discriminator loss contains two competing negative log terms:

$$-\log d(x) \quad \text{and} \quad -\log(1 - d(g(z))).$$

These two terms must stay in balance. If the Discriminator becomes too strong, then

$$d(x) \approx 1, \quad d(g(z)) \approx 0,$$

which causes the Generator to receive almost no gradient. If the Discriminator is too weak, it provides no meaningful signal to the Generator. Thus, the training dynamics are highly unstable.

The term

$$-\log(1 - d(g(z)))$$

saturates when the Discriminator confidently rejects fake samples:

$$d(g(z)) \approx 0 \quad \Rightarrow \quad \log(1 - d(g(z))) \approx 0.$$

This means the Generator receives extremely small gradients at the beginning of training, causing slow or stalled learning. This gradient saturation is one of the most fundamental weaknesses of GANs.

The fake-data term in the Discriminator loss,

$$-\log(1 - d(g(z))),$$

encourages the Generator to find any configuration that fools the Discriminator. Because the loss does not penalize lack of diversity, the Generator may learn:

$$g(z_1) \approx g(z_2) \approx \cdots ,$$

resulting in **mode collapse**. The Generator outputs a small set of repetitive samples instead of covering the true data distribution.

GAN optimization is a two-player game and the Discriminator loss involves competing terms:

$$-\log d(x), \quad -\log(1 - d(g(z))).$$

These objectives pull the parameters in different directions, often creating:

- cyclic gradient behavior,
- absence of stable equilibrium,
- non-convergent training trajectories.

This makes GANs significantly harder to train than likelihood-based generative models.

Latent Variable z

In standard Generative Adversarial Networks (GANs), the latent variable z is sampled from a fixed prior distribution:

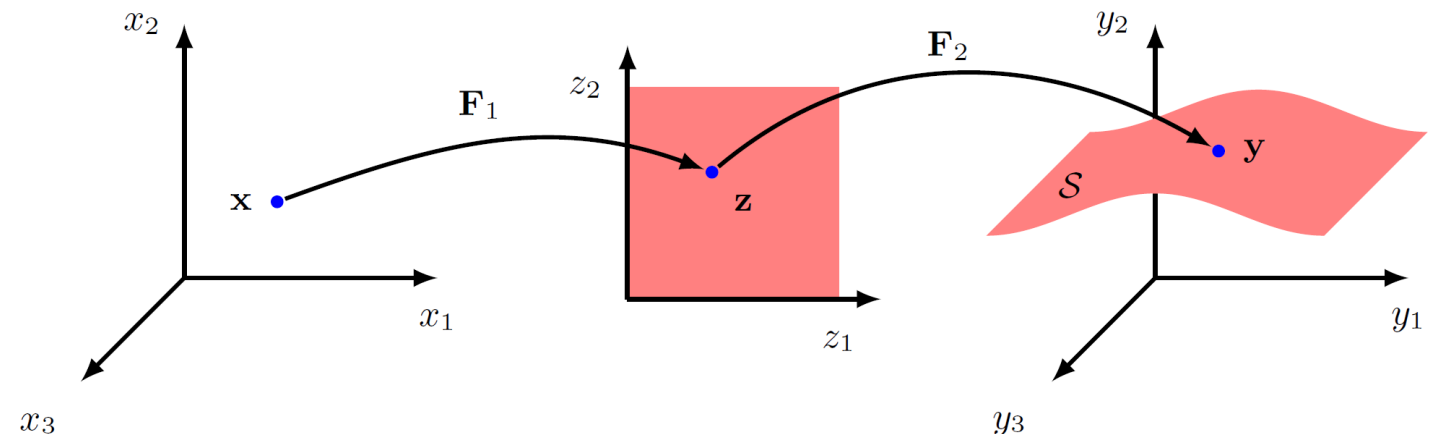
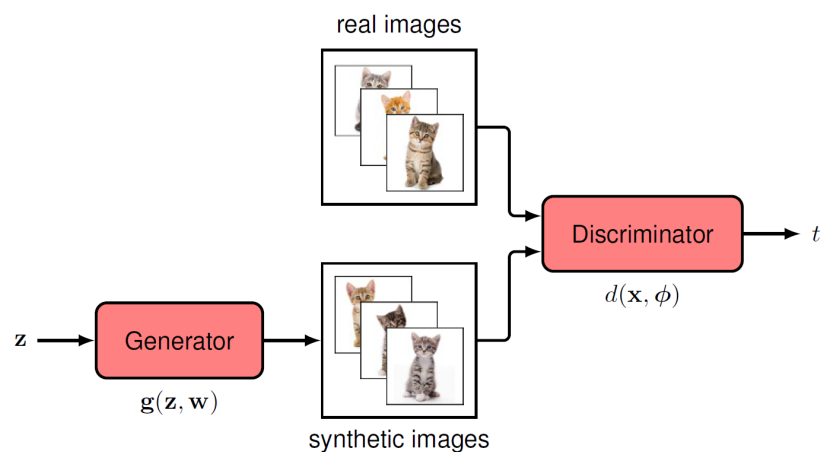
$$z \sim p(z),$$

typically a standard Gaussian distribution:

$$z \sim \mathcal{N}(0, I).$$

Thus, z serves as **random noise** that the Generator transforms into an output in the data space. It is not learned, not updated, and has no trainable parameters.

During GAN training, only the **Generator parameters w** and the **Discriminator parameters ϕ** are updated based on gradients.

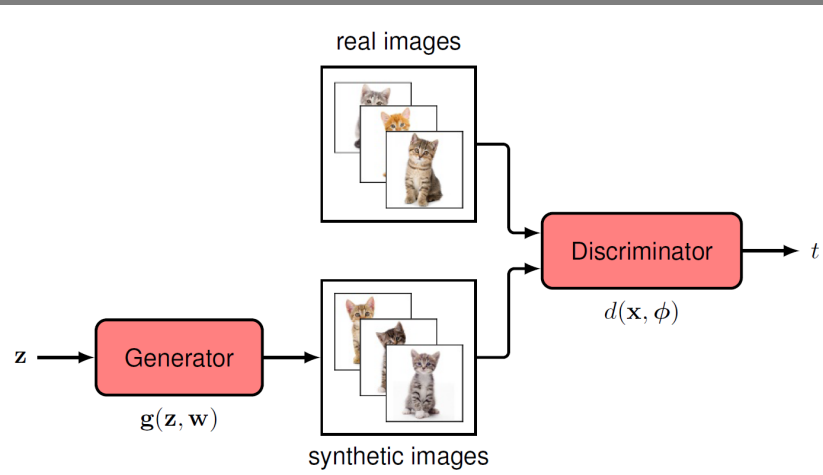


Why is Latent Variable z NOT Updated ?

If GAN training attempted to optimize z , the latent space would lose its consistency, and the model could:

- overfit to specific samples,
- lose generative diversity,
- fail to produce new images during inference.

A fixed prior distribution maintains a stable structure in the latent space and supports generalization.



Do Any Models Learn or Optimize z ?

Although standard GANs do not update z , some other generative models treat latent variables differently:

1. Variational Autoencoders (VAEs)

VAEs learn an approximate posterior distribution:

$$q(z|x) = \mathcal{N}(\mu(x), \sigma(x)).$$

Here, z is inferred for each input and plays a central role in reconstruction.

2. Diffusion Models

Diffusion models do not optimize a latent variable z directly. However, latent diffusion models (e.g., Stable Diffusion) operate in a latent representation created by a VAE encoder.

3. StyleGAN

StyleGAN samples z from a prior and maps it to an intermediate latent w , but z itself is still not learned.

4. Latent Optimization Methods

Some GAN variants optimize z at inference time for tasks such as image editing or inversion, but this is not part of standard GAN training.

VAE (Variational AutoEncoder)

A Variational Autoencoder (VAE) is a generative model that combines neural networks with techniques from Bayesian statistics. Its name comes from the combination of two ideas:

- **Autoencoder:** a neural network that learns to encode and decode data.
- **Variational:** referring to *variational inference*, a method used to approximate probability distributions.

A Variational Autoencoder (VAE) is a neural network architecture that can compress images into a latent representation and reconstruct them back into full-resolution images. Diffusion models such as Stable Diffusion use a VAE to operate in a smaller latent space instead of full pixel space, drastically reducing computation.

The “Autoencoder” component in VAE

A VAE has the same overall structure as a classical autoencoder:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}.$$

However, unlike a traditional autoencoder:

- The encoder does not output a single point z .
- Instead, it outputs parameters of a probability distribution:

$$q(z|x) = \mathcal{N}(z; \mu(x), \sigma^2(x)).$$

A sample is then drawn from this distribution:

$$z \sim q(z|x),$$

and the decoder reconstructs the input from z :

$$\hat{x} = p(x|z).$$

Thus, the architecture functions like an autoencoder, but with a probabilistic latent space.

The “Variational” component in VAE

The term “variational” refers to **variational inference**, a method for approximating an intractable posterior distribution.

The true posterior $p(z|x)$ is typically impossible to compute directly. Instead, the VAE introduces a tractable approximation:

$$q(z|x),$$

and optimizes it by minimizing the KL divergence:

$$D_{\text{KL}}(q(z|x) \parallel p(z|x)) .$$

Because we cannot compute $p(z|x)$ exactly, we maximize the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}(x) = \mathbb{E}_{z \sim q(z|x)} [\log p(x|z)] - D_{\text{KL}}(q(z|x) \parallel p(z)) .$$

Maximizing this quantity is the essence of variational inference.

Thus, the model is called **variational** because it uses techniques from variational Bayesian learning.

A VAE consists of two main components: an encoder and a decoder.

Encoder

The encoder converts an image into a latent vector z . Unlike a standard autoencoder, a VAE outputs two vectors:

$$\mu(x), \quad \sigma(x),$$

representing the mean and variance of a Gaussian distribution. The latent vector is sampled using the reparameterization trick:

$$z = \mu(x) + \sigma(x) \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

This introduces controlled randomness, enabling the model to learn a smooth generative latent space.

Decoder

The decoder reconstructs the original image from the latent vector:

$$\hat{x} = \text{Decoder}(z).$$

It learns to generate realistic images from the compressed representation.

Unlike standard autoencoders, VAEs force the latent space to follow a known prior distribution (typically a Gaussian). This ensures:

- **Smooth Latent Space:** Nearby points produce similar images.
- **Stable Sampling:** Random samples from the latent space generate realistic outputs.
- **Continuity:** Interpolation in latent space is meaningful.

This structure is essential for generative modeling.

Why Diffusion Models need a VAE ?

Diffusion in Pixel Space is Expensive

A 512×512 RGB image contains:

$$512 \times 512 \times 3 = 786,432 \text{ values,}$$

which makes diffusion extremely computationally costly.

VAE Compresses the Image

Stable Diffusion's VAE encoder reduces the resolution by a factor of 8:

$$512 \times 512 \longrightarrow 64 \times 64 \times 4.$$

Diffusion then operates in this *latent space*, which is much smaller.
Benefits:

- Reduced GPU memory usage,
- Faster training and inference,
- Feasible high-resolution generation.

Input: Training data set $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$
Encoder network $\{\mu_j(\mathbf{x}_n, \phi), \sigma_j^2(\mathbf{x}_n, \phi)\}, \quad j \in \{1, \dots, M\}$
Decoder network $\mathbf{g}(\mathbf{z}, \mathbf{w})$
Initial weight vectors $\mathbf{w}, \phi$
Learning rate η

Output: Final weight vectors $\mathbf{w}, \phi$

```
repeat
   $\mathcal{L} \leftarrow 0$ 
  for  $j \in \{1, \dots, M\}$  do
     $\epsilon_{nj} \sim \mathcal{N}(0, 1)$ 
     $z_{nj} \leftarrow \mu_j(\mathbf{x}_n, \phi)\epsilon_{nj} + \sigma_j^2(\mathbf{x}_n, \phi)$ 
     $\mathcal{L} \leftarrow \mathcal{L} + \frac{1}{2} \{1 + \ln \sigma_{nj}^2 - \mu_{nj}^2 - \sigma_{nj}^2\}$ 
  end for
   $\mathcal{L} \leftarrow \mathcal{L} + \ln p(\mathbf{x}_n | \mathbf{z}_n, \mathbf{w})$ 
   $\mathbf{w} \leftarrow \mathbf{w} + \eta \nabla_{\mathbf{w}} \mathcal{L}$  // Update decoder weights
   $\phi \leftarrow \phi + \eta \nabla_{\phi} \mathcal{L}$  // Update encoder weights
until converged
return  $\mathbf{w}, \phi$ 
```

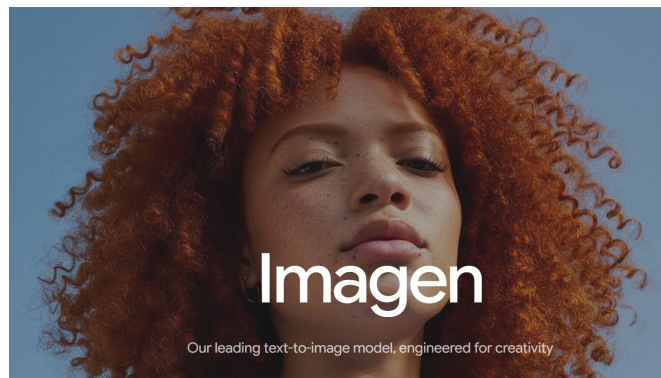
Diffusion Models

Diffusion models have become one of the most powerful and influential classes of generative models in modern machine learning.

They are the foundation of systems such as Stable Diffusion, DALL·E 2, Imagen, and many recent models for image, audio, and video synthesis.

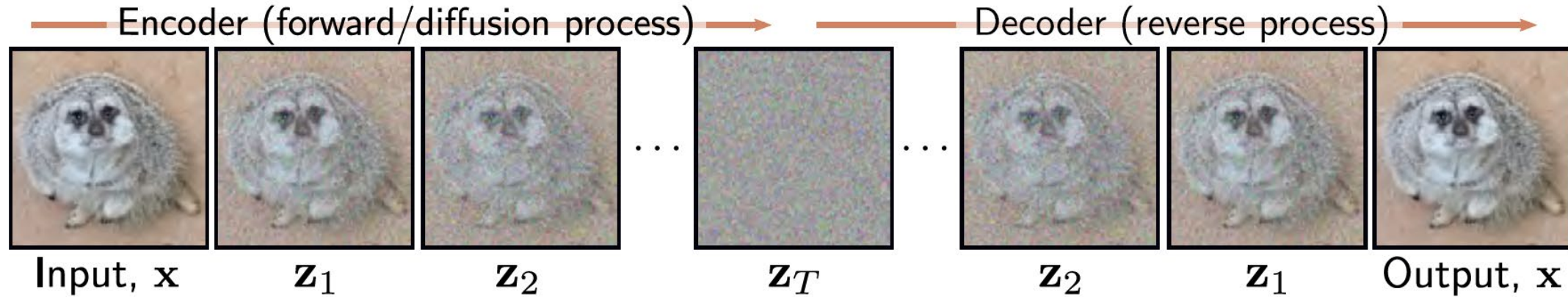
Diffusion models currently achieve state-of-the-art results in many domains:

- high-resolution image generation,
- text-to-image synthesis,
- super-resolution and image editing,
- audio and speech synthesis,
- 3D and video generation.



Compared to GANs, diffusion models produce more diverse and stable samples and avoid adversarial training instability.

Diffusion Models



Diffusion models work in two main phases:

1. **Forward (noising) process:** Gradually add noise to an image until it becomes almost pure noise.
2. **Reverse (denoising) process:** Learn how to remove the noise step by step, turning noise back into a clean image.

Diffusion Models

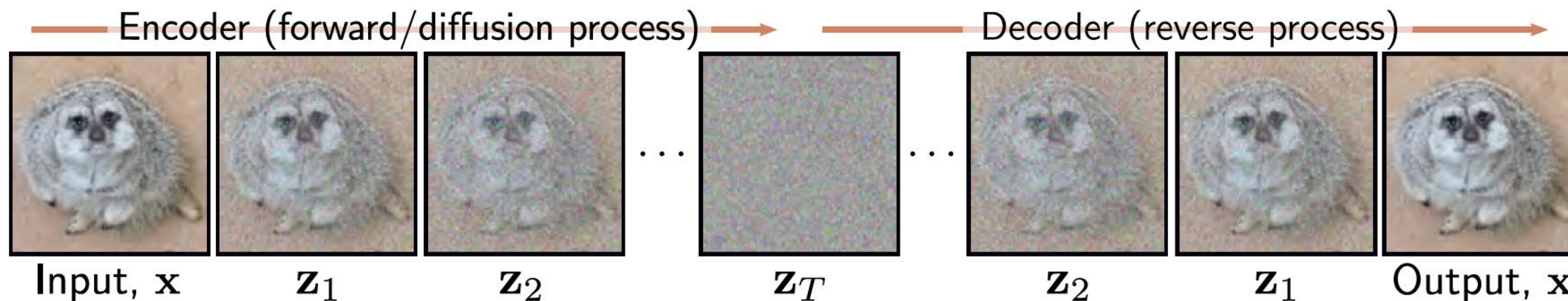


Figure 18.1 Diffusion models. The encoder (forward, or diffusion process) maps the input $\mathbf{x}$ through a series of latent variables $\mathbf{z}_1 \dots \mathbf{z}_T$. This process is pre-specified and gradually mixes the data with noise until only noise remains. The decoder (reverse process) is learned and passes the data back through the latent variables, removing noise at each stage. After training, new examples are generated by sampling noise vectors $\mathbf{z}_T$ and passing them through the decoder.

A diffusion model is a type of generative model that learns how to create new data (for example, images) by *learning to remove noise*. You can think of it as a smart denoising machine.

1. Forward (Noising) Process

Start with a real input image x . At each step, we add a small amount of Gaussian noise:

$$x \rightarrow z_1 \rightarrow z_2 \rightarrow \cdots \rightarrow z_T,$$

where:

- x is the original clean image,
- z_T is almost pure noise,
- each z_t is a slightly noisier version of the previous step.

This forward diffusion process is fixed and does not need to be learned.

2. Reverse (Denoising) Process

The reverse process tries to undo the noising steps:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_1 \rightarrow x.$$

A neural network learns how to remove the noise from each z_t .

Noise Prediction View

Instead of directly predicting the clean image, the network predicts the noise in z_t .

- Input: a noisy latent variable z_t and the timestep t .
- Output: predicted noise $\epsilon_\theta(z_t, t)$.

To denoise:

$$\text{cleaner step} = z_t - \epsilon_\theta(z_t, t).$$

How Do We Generate a New Image ?

To generate a new image:

1. Start with pure noise z_T .
2. Gradually denoise:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_1 \rightarrow x.$$

3. The final output is a new generated sample x .

No input image is needed; the model starts from noise.

Why is it Called “Diffusion”?

The forward process spreads (diffuses) information by repeatedly adding noise, similar to how ink spreads in water. The reverse process learns to reverse this diffusion.

Key Intuition in Diffusion Models

- Adding noise is easy.
- Learning to remove noise gradually is powerful.
- By reversing the noise process step by step, the model learns to generate realistic data.

1. Forward Diffusion Process (Adding Noise)

We start with a clean input image x . Noise is gradually added over T steps, producing latent variables $z_1, z_2, \dots, z_T$:

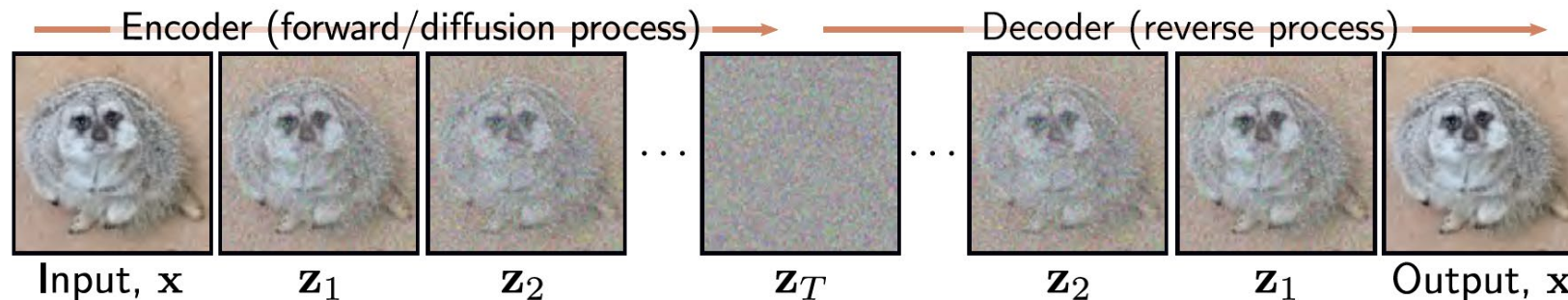
$$x = z_0 \rightarrow z_1 \rightarrow z_2 \rightarrow \dots \rightarrow z_T.$$

The forward process is defined as a simple Gaussian transition:

$$q(z_t | z_{t-1}) = \mathcal{N}\left(z_t; \sqrt{1 - \beta_t} z_{t-1}, \beta_t I\right),$$

where:

- β_t controls how much noise is added at step t ,
- usually $0 < \beta_1 < \beta_2 < \dots < \beta_T \ll 1$.



1. Forward Diffusion Process (Adding Noise)

Closed-form Noising

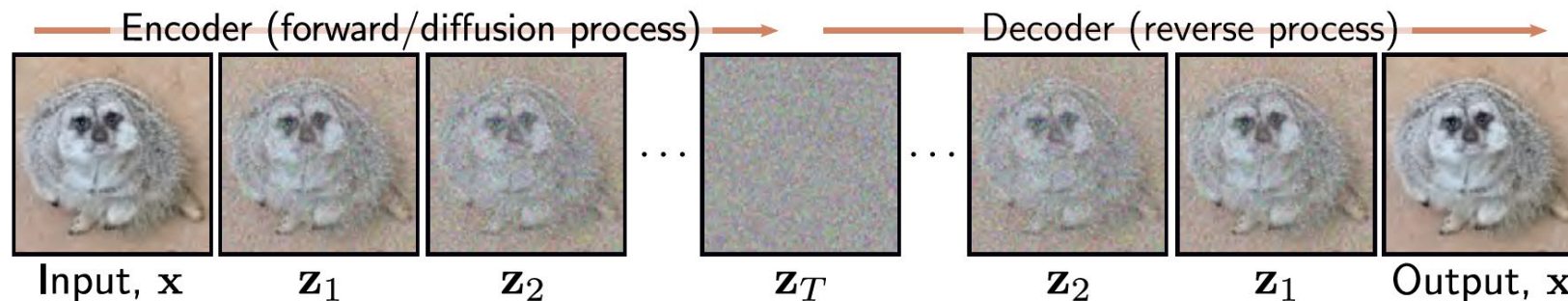
Instead of applying noise step-by-step, we can directly sample z_t from x :

$$z_t = \sqrt{\alpha_t} x + \sqrt{1 - \alpha_t} \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

where:

$$\alpha_t = \prod_{s=1}^t (1 - \beta_s).$$

This expresses z_t as a mixture of the clean image and Gaussian noise.



2. Reverse Diffusion Process (Removing Noise)

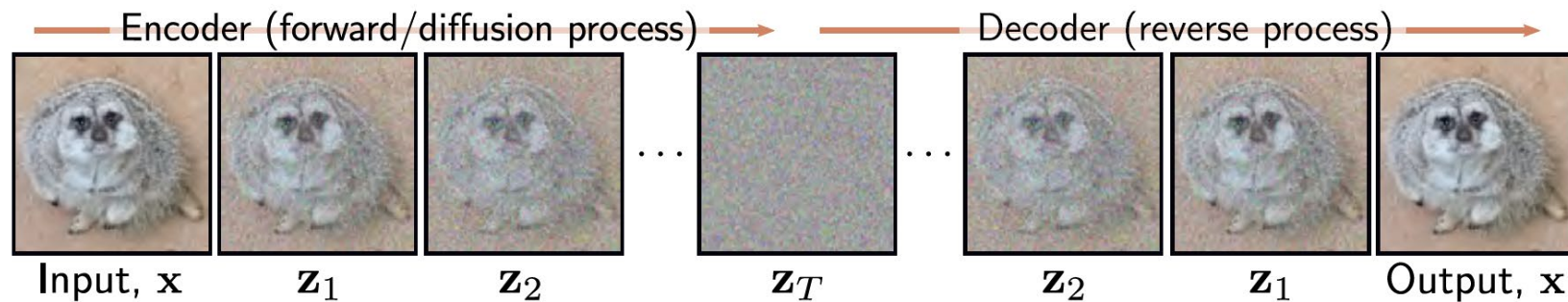
In generation, we want to reverse the process:

$$z_T \rightarrow z_{T-1} \rightarrow \cdots \rightarrow z_1 \rightarrow x.$$

The true reverse distribution:

$$q(z_{t-1} \mid z_t)$$

is intractable. So DDPM trains a neural network to approximate it.



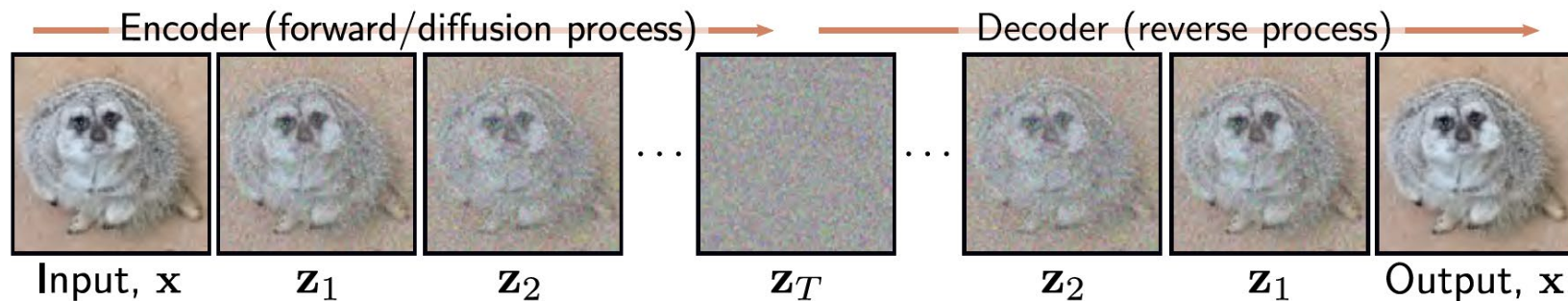
3. Noise Prediction Model

Instead of predicting the clean image directly, the neural network predicts the noise in z_t :

$$\epsilon_{\theta}(z_t, t).$$

Given the noise prediction, we estimate the clean image:

$$\hat{x}_{\theta}(z_t, t) = \frac{1}{\sqrt{\alpha_t}} (z_t - \sqrt{1 - \alpha_t} \epsilon_{\theta}(z_t, t)).$$



4. Reverse Step Distribution

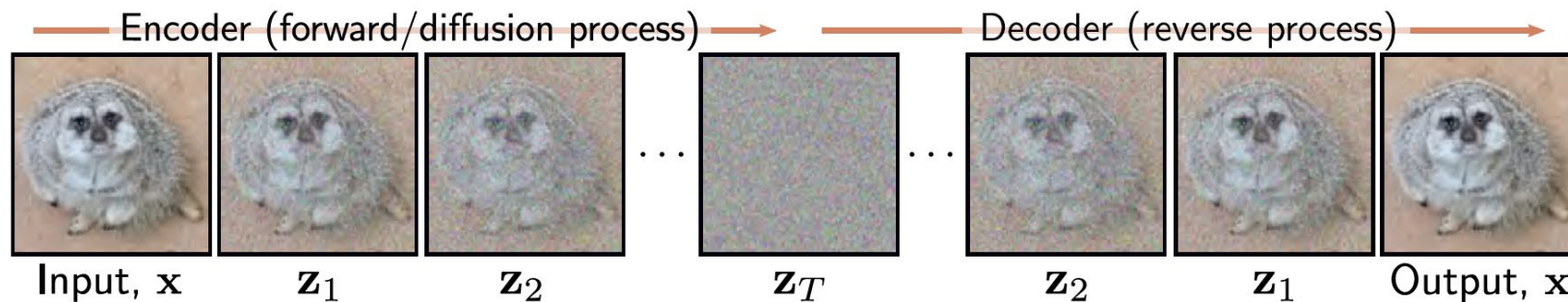
The learned reverse distribution is modeled as:

$$p_{\theta}(z_{t-1} \mid z_t) = \mathcal{N}(z_{t-1}; \mu_{\theta}(z_t, t), \Sigma_t),$$

where the mean is:

$$\mu_{\theta}(z_t, t) = \frac{1}{\sqrt{1 - \beta_t}} \left(z_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(z_t, t) \right),$$

and Σ_t is typically a fixed variance.

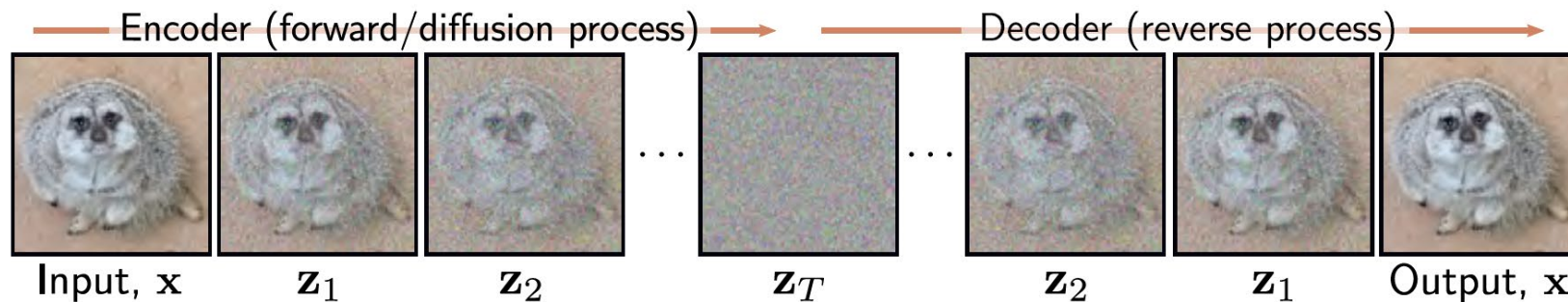


5. Training Objective (Simplified Loss)

DDPM uses a very simple objective:

$$\mathcal{L} = \mathbb{E}_{x, t, \epsilon} [\|\epsilon - \epsilon_{\theta}(z_t, t)\|^2] .$$

This trains the model to predict the noise added at each timestep.



DDPM (Denoising Diffusion Probabilistic Models)

Input: Training data $\mathcal{D} = \{\mathbf{x}_n\}$

Noise schedule $\{\beta_1, \dots, \beta_T\}$

Output: Network parameters $\mathbf{w}$

for $t \in \{1, \dots, T\}$ **do**

$\alpha_t \leftarrow \prod_{\tau=1}^t (1 - \beta_\tau)$ // Calculate alphas from betas

end for

repeat

$\mathbf{x} \sim \mathcal{D}$ // Sample a data point

$t \sim \{1, \dots, T\}$ // Sample a point along the Markov chain

$\epsilon \sim \mathcal{N}(\epsilon | \mathbf{0}, \mathbf{I})$ // Sample a noise vector

$\mathbf{z}_t \leftarrow \sqrt{\alpha_t} \mathbf{x} + \sqrt{1 - \alpha_t} \epsilon$ // Evaluate noisy latent variable

$\mathcal{L}(\mathbf{w}) \leftarrow \|\mathbf{g}(\mathbf{z}_t, \mathbf{w}, t) - \epsilon\|^2$ // Compute loss term

 Take optimizer step

until converged

return $\mathbf{w}$

6. Sampling (Generating New Images)

To generate a new image:

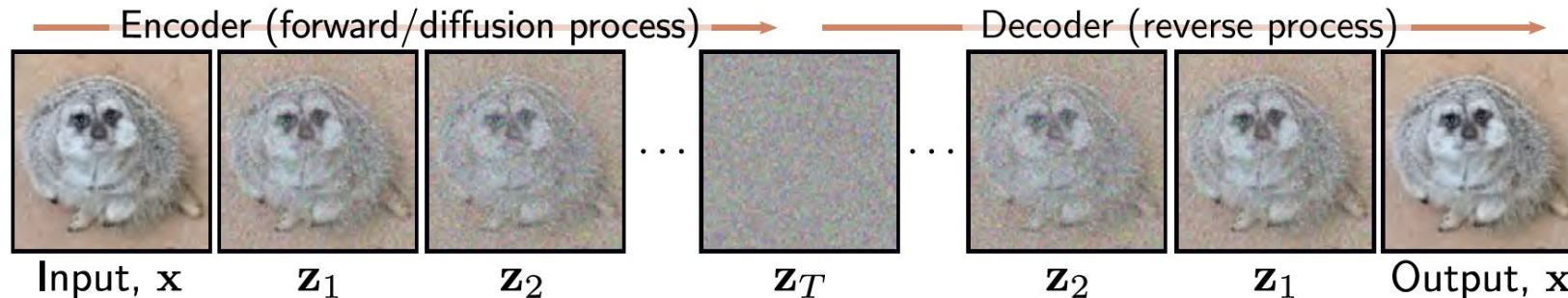
1. Start from pure Gaussian noise:

$$z_T \sim \mathcal{N}(0, I).$$

2. For $t = T, T - 1, \dots, 1$, compute:

$$z_{t-1} = \mu_{\theta}(z_t, t) + \sqrt{\Sigma_t} \eta, \quad \eta \sim \mathcal{N}(0, I).$$

3. The final output z_0 is the generated sample x .

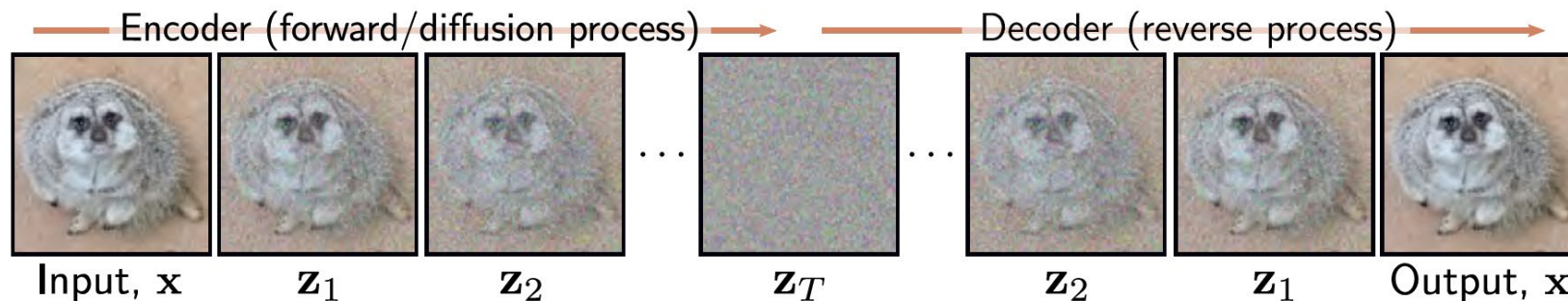


DDPM (Denoising Diffusion Probabilistic Models)

- Forward process: Simple Gaussian noise is added to the input x .
- Reverse process: A neural network learns how to remove noise.
- The model predicts noise, not pixels.
- Images are generated by starting from noise and denoising step-by-step.

DDPMs combine:

- a simple forward noising process,
- a learned reverse denoising process,
- and a clean mathematical description based on Gaussian transitions.



Thank you!

